

Análisis y diseño con patrones

PEC1: Principios de diseño y patrones de análisis

Índice

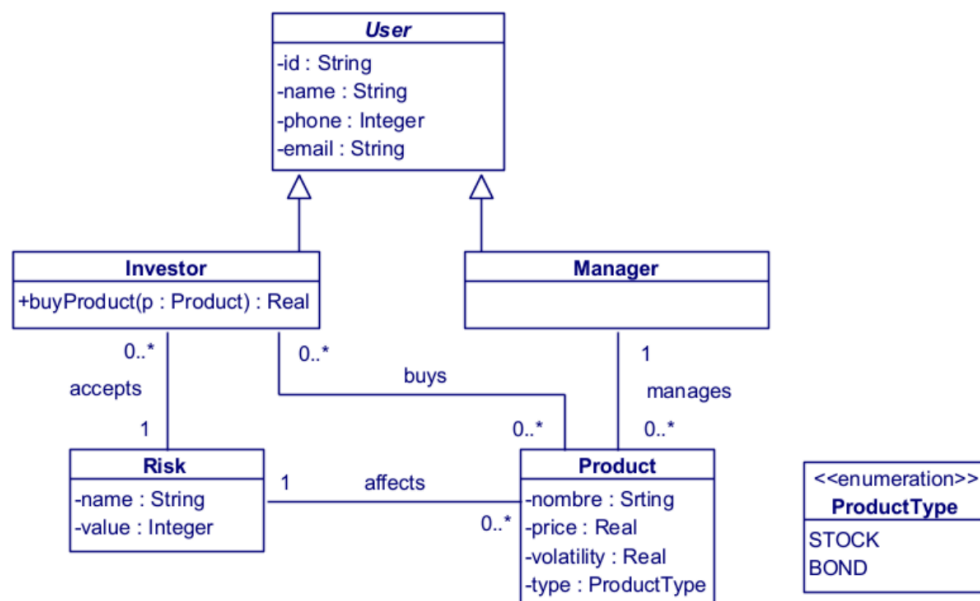
Contexto.....	2
Pregunta 1 (30%).....	3
Pregunta 1.1 (5 %).....	4
Pregunta 1.2 (15 %).....	5
Pregunta 1.3 (10 %).....	7
Pregunta 2 (20 %).....	9
Pregunta 2.1 (5 %).....	9
Pregunta 2.2 (15 %).....	10
Pregunta 3 (30 %).....	12
Pregunta 3.1. (5 %).....	12
Pregunta 3.2 (15 %).....	13
Pregunta 3.3 (10 %).....	14
Pregunta 4 (20 %).....	16
Pregunta 4.1. (2.5 %).....	17
Pregunta 4.2. (2.5 %).....	17
Pregunta 4.3. (15 %).....	18

Contexto

Todos los ejercicios de esta actividad están relacionados con un hipotético sistema web para realizar inversiones financieras llamado InvestUOC y que permitirá a los usuarios gestionar sus carteras de inversión. Tras las reuniones con los analistas, se han definido los siguientes requisitos iniciales:

Inversores y gestores. El sistema tiene usuarios inversores (*Investors*) y usuarios gestores (*Managers*). De cada uno de ellos nos interesa conocer el identificador, el nombre, el teléfono y su email. Los gestores se encargan de administrar los diferentes productos financieros del sistema y los inversores, o clientes del sistema, son los que adquieren o compran los diferentes productos. Los inversores tienen asignado un nivel de riesgo, en base a un cuestionario que se les hace al darse de alta.

Productos de inversión. Los productos de inversión (*Product*) corresponden con todos los productos financieros que los inversores pueden comprar a través de la plataforma. Por cada producto, el sistema registra el nombre del producto, el precio, la volatilidad y el tipo del producto (acciones o bonos). Adicionalmente, cada producto tiene un nivel de riesgo (*Risk*). El riesgo dispone de un nombre y de un valor numérico. A mayor valor, mayor riesgo.



Restricciones de integridad:

- “User.id” es el identificador del usuario.
- “Risk.name” es el identificador del riesgo.
- “Risk.Value”: Solo puede tomar valores de 1 a 7, que son las categorías de riesgos establecidas a nivel internacional.
- “Product.name” es el identificador de producto.

Partiremos de un sistema/modelo básico al que se le irán añadiendo nuevas funcionalidades a lo largo de los ejercicios que vosotros tenéis que ir resolviendo.

Pregunta 1 (30%)

En este ejercicio revisaremos los principios de diseño y las violaciones de los mismos en el código.

Una de las funciones más importantes del sistema InvestUOC es la compra de activos financieros por parte de los inversores, y esto es posible gracias al método *buyProduct()* de la clase *Investor*. Al comprar un producto, se debe validar que el riesgo del producto es igual o inferior al riesgo que el inversor está dispuesto a asumir; de este modo se verifica que el inversor no pueda comprar cosas que no encajen con su perfil.

El modelo de negocio de InvestUOC es cobrar una comisión por cada uno de los productos que compran los inversores. Actualmente, la plataforma ofrece acciones (*stocks*) y bonos (*bonds*) que tienen una comisión del 2% y 1%, respectivamente. El método *buyProduct()* devuelve el precio de venta del producto teniendo en cuenta la comisión.

A continuación, se adjunta el pseudocódigo del subsistema descrito en el enunciado.

```
public class Investor extends User {
    private Risk userRisk; // Riesgo de inversión que acepta el inversor
    public void User(...) {}

    /* getters and setters ... */

    public Real buyProduct(Product p) {
        // Validación de riesgo: El inversor comprueba si el producto le encaja
        if (p.getRisk().getValue() > this.userRisk.getValue()) {
            throw new Exception("El producto que está tratando de comprar " +
                                "no es adecuado para el perfil de riesgo del" +
                                " inversor");
        }

        // Cálculo de comisión según el tipo de producto
        Real total = p.getPrice();
        if (p.getType() == ProductType.STOCK) {
            total = total + (total * 0.02); // 2% comisión para acciones
        } else if (p.getType() == ProductType.BOND) {
            total = total + (total * 0.01); // 1% comisión para bonos
        }
        this.registerPurchase(p);
        return total;
    }
}
```

Pregunta 1.1 (5 %)

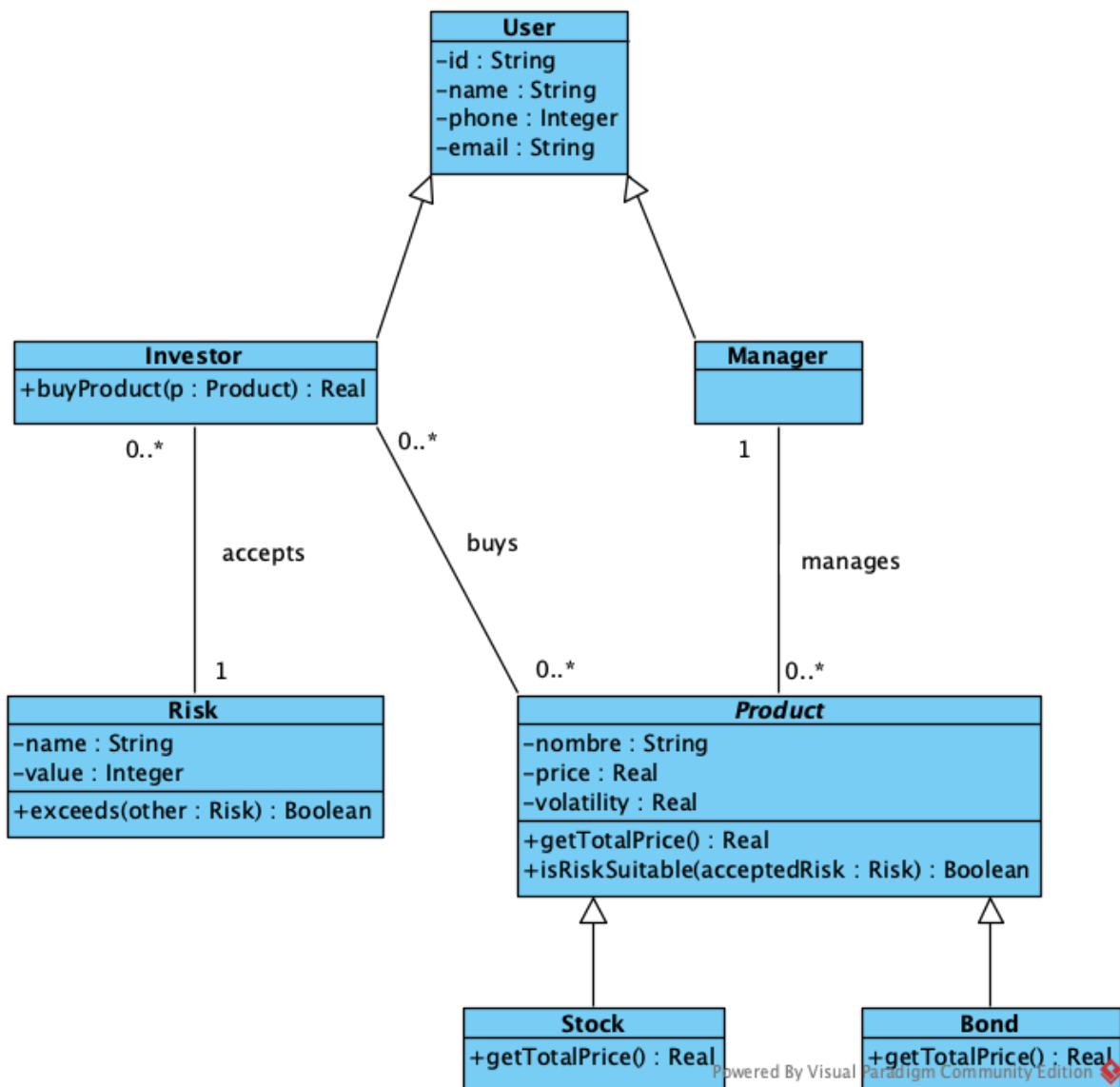
Enumera y explica los principios de diseño que no se cumplen en la descripción y pseudocódigo proporcionados en el ejercicio 1.

En base al código de la clase *Investor* dado en el enunciado, podemos comprobar que no se respetan los siguientes principios de diseño:

- **Bajo acoplamiento (*Low coupling*):** La clase *Investor* está fuertemente acoplada a múltiples conceptos ajenos a ella. Conoce la existencia de un enumerado *ProductType* (y sus valores *STOCK*, *BOND*), conoce las tasas de comisión exactas (0.02, 0.01), y conoce la estructura interna del riesgo (*Risk*). Este acoplamiento va en contra de un buen diseño en el que las clases estarían lo más independientes posible. Actualmente, cualquier cambio en las reglas de negocio de los productos (por ejemplo, cambiar la comisión de un bono al 1,5%) , implicaría cambios en la clase *Investor*, cuando lógicamente un inversor no debería verse afectado por una decisión de precios de la plataforma.
- **Alta cohesión (*High cohesion*):** El método *buyProduct* hace demasiadas cosas inconexas, como validar riesgos, realizar cálculos matemáticos de finanzas y registrar operaciones. Por ello, vemos que la clase *Investor* tiene una cohesión baja por incluir lógica de negocio que le corresponde al sistema de ventas o a los propios productos. Su única responsabilidad aquí debería ser iniciar la intención de compra y registrar el producto en la cartera del inversor al procesar la transacción.
- **Abierto-cerrado (OCP, *Open-closed principle*):** El software debe estar abierto a la extensión pero cerrado a la modificación. Si mañana InvestUOC añade un producto tipo con una comisión determinada, tendríamos que modificar el método *buyProduct* para añadir un nuevo *else if*. Esto rompe el OCP, ya que lo correcto sería que el producto expusiera un método (ej. *getCommission()*) que cada tipo de producto implementase de forma diferente.
- **Ley de Demeter (*Law of Demeter*):** Este principio, dicta que un objeto solo debe invocar métodos de sus componentes directos. En este caso, *Investor* se comunica con *Product* (*p*) para pedirle un objeto de la clase *Risk*, para luego pedirle a ese objeto *Risk* su valor numérico (*getValue()*), lo cual genera una dependencia de la estructura interna del producto.

Pregunta 1.2 (15 %)

Proporciona un modelo de clases UML que corrija los principios que no se cumplen según el apartado anterior; y explica brevemente el porqué de los cambios realizados. No hace falta explicar detalladamente cada elemento del diagrama, solamente los cambios principales.



Vamos a desglosar los cambios realizados en función de los principios de diseño corregidos:

- **Abierto-cerrado (OCP, Open-closed principle) y alta cohesión:** Hemos eliminado la enumeración *ProductType* y hemos convertido la clase *Product* en abstracta con el fin de crear dos subclases nuevas, *Stock* y *Bonds*, las cuales heredan de *Product*. Gracias al método abstracto *getTotalPrice()* en *Product*, delegamos el cálculo de la comisión a cada tipo de producto específico. Si el día de mañana InvestUOC quiere añadir otro tipo de producto, solo hay que crear una nueva subclase para dicho producto sin modificar el código de la clase *Investor*. De esta forma, el sistema queda abierto a extensión pero cerrado a la modificación.

Además, mejora la cohesión, ya que *Investor* deja de ser responsable de calcular comisiones financieras.

- **Ley de Demeter y bajo acoplamiento:** Hemos añadido métodos de validación como *isRiskSuitable(acceptedRisk: Risk)* y *exceeds(other: Risk): Boolean*, ya que, de la forma en la que estaba antes, el inversor accedía a las propiedades de otros objetos para obtener el valor del riesgo asociado al producto. Esta práctica viola la ley de Demeter, por lo que la validación mediante los nuevos métodos oculta la lógica correspondiente a las clases que deben encargarse de ello (*Risk* y *Product*) y reduce drásticamente el acoplamiento entre *Investor* y los detalles de implementación de *Risk*.

Pregunta 1.3 (10 %)

Proporciona el pseudocódigo que corrige los principios que no se cumplen según la pregunta 1.1 y que sea coherente con el nuevo modelo de la pregunta 1.2.

```
public class Risk {
    private String name;
    private Integer value;

    public boolean exceeds(Risk otherRisk) {
        return this.value > otherRisk.value;
    }
}

public abstract class Product {
    protected String nombre;
    protected Real price;
    protected Real volatility;
    protected Risk productRisk;

    public abstract Real getTotalPrice();

    public boolean isRiskSuitable(Risk investorRisk) {
        return !this.productRisk.exceeds(investorRisk);
    }
}

public class Stock extends Product {
    public Real getTotalPrice() {
        return this.price + (this.price * 0.02); // 2% de comisión
    }
}

public class Bond extends Product {
    public Real getTotalPrice() {
        return this.price + (this.price * 0.01); // 1% de comisión
    }
}

public class Investor extends User {
    private Risk userRisk;

    public void User(...) {}
}
```

```
/* getters and setters ... */

public Real buyProduct(Product p) {
    if (!p.isRiskSuitable(this.userRisk)) {
        throw new Exception("El producto que está tratando de comprar " +
            "no es adecuado para el perfil de riesgo del
inversor");
    }

    Real total = p.getTotalPrice();
    this.registerPurchase(p);
    return total;
}

private void registerPurchase(Product p) {
    // Lógica interna para añadir a la cartera...
}
}
```

Pregunta 2 (20 %)

Tras el éxito inicial, la plataforma InvestUOC ha decidido internacionalizar su catálogo, permitiendo la comercialización de activos financieros procedentes de diversas regiones geográficas. Los productos pueden cotizar en monedas diferentes al euro (EU), específicamente en dólares (USD) y libras esterlinas (GBP). El sistema debe ser capaz de gestionar los precios teniendo en cuenta la moneda.

La gerencia ha observado que algunas operaciones generan beneficios que ni siquiera cubren el coste de la operación (márgenes "irrisorios"), mientras que otras imponen recargos que los inversores consideran "abusivos". Se necesita la implementación de una estructura que defina una horquilla de rentabilidad para las comisiones que se aplican a cada producto. Las rentabilidades deben expresarse en valor absoluto y no en porcentaje, y deben tener en cuenta las diferentes regiones geográficas. Es vital que el diseño permita cambiar la horquilla de las comisiones sin que esto afecte a la estructura de la clase *Product*, asegurando que la lógica de validación de magnitudes no quede dispersa por el sistema.

Pregunta 2.1 (5 %)

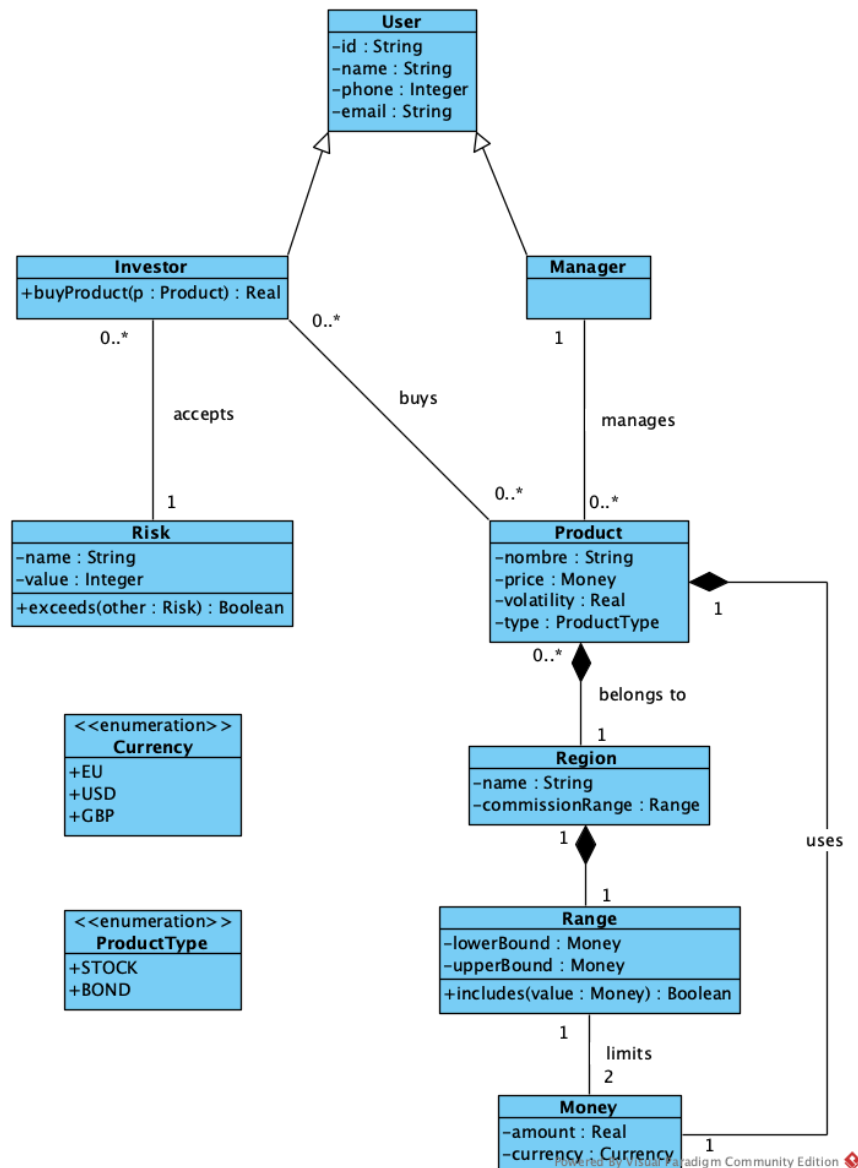
Identificar qué patrones de análisis deben aplicarse para diseñar el sistema, justificando brevemente su utilidad.

Para resolver los requisitos planteados en la internacionalización y gestión de comisiones de InvestUOC, deben aplicarse estos patrones de análisis:

- **Patrón cantidad (*quantity*):** El sistema debe gestionar precios y rentabilidades expresadas en valores absolutos y en diversas divisas. Para centralizar la lógica matemática y la validación de magnitudes, este patrón elimina el riesgo de usar tipos primitivos sueltos, ya que agrupa en un único objeto el valor numérico (la magnitud) y su unidad monetaria (la divisa). De esta forma, se evitan errores críticos como sumar directamente euros con dólares.
- **Patrón rango (*range*):** El enunciado determina que se necesita definir una horquilla de rentabilidad acotada por un límite inferior para evitar márgenes irrisorios y un límite superior para evitar recargos abusivos. Al aplicar este patrón, se encapsulan los límites superior e inferior de la horquilla en un único objeto en vez de tener atributos separados y lógica condicional dispersa. Su utilidad principal es ofrecer métodos propios, como verificar si un valor está incluido en el rango, para validar si el beneficio de una operación es correcto.
- **Patrones de asignación de responsabilidades:** Para cumplir con el requisito de que la horquilla varíe según la región geográfica sin afectar a la estructura de la clase *Product*, la solución pasa por la delegación de responsabilidades. La clase *Product* debe asociarse a una clase *Region*, la cual contendrá el atributo de tipo *Range*. De esta manera, si cambian las políticas de comisiones para una zona concreta, simplemente se actualiza el objeto *Range* dentro de su *Region* correspondiente. Así, la clase *Product* delega la validación y no sufre ninguna modificación estructural.

Pregunta 2.2 (15 %)

Proponer un diagrama estático de análisis, aplicando los patrones identificados. Usa de base el diagrama del enunciado; puedes dibujar solo la parte del modelo necesaria para explicar el ejercicio. No hace falta explicar detalladamente cada elemento del diagrama, solamente los cambios principales.



Vamos a desglosar los cambios realizados en función de los principios de diseño corregidos:

- **Aplicación del patrón cantidad (*quantity*):** Hemos sustituido el tipo *Real* del atributo *price* en la clase *Product* por una clase nueva *Money*. La clase *Money* encapsula el valor numérico (*amount: Real*) y la unidad monetaria delegándola a una nueva enumeración *Currency*, la cual tiene los valores EU, USD y GBP. De esta forma, evitamos la dispersión de monedas y errores matemáticos.

- **Aplicación del patrón rango (*range*):** Hemos creado la clase *Range* para definir la horquilla de rentabilidad de las comisiones. Tiene dos atributos (*lowerBound* y *upperBound*) que también son de tipo *Money*, ya que el enunciado detalla que sean valores absolutos e internacionalizados, no porcentajes.
- **Aplicación de patrones de asignación de responsabilidades:** Para evitar que la estructura de *Product* se vea afectada por los cambios de comisiones, añadimos la clase *Region*. *Product* tiene una asociación de multiplicidad 0..* a 1 con *Region*, la cual manifiesta que cada producto pertenece a una región. La clase *Region* que contiene y gestiona la asociación con *Range* (*commissionRange*), de modo que, si la directiva decide cambiar los límites de los márgenes abusivos o irrisorios de los mercados que operan en dólares, simplemente se actualiza el objeto *Range* asociado a la *Region* de Norteamérica, sin necesidad de modificar la lógica interna de *Product*.

Pregunta 3 (30 %)

Debido a la creciente demanda de diversificación, InvestUOC ha decidido evolucionar su catálogo de servicios mediante la introducción de Instrumentos Financieros Estructurados. A diferencia de los activos básicos (como acciones o bonos), estos instrumentos operan como vehículos de agregación que permiten combinar diferentes activos bajo una única denominación comercial.

Una particularidad crítica de estos nuevos instrumentos es su naturaleza recursiva: un instrumento estructurado puede estar conformado por un conjunto de activos individuales, pero el sistema también debe permitir que una estrategia de inversión incorpore otras estrategias ya existentes de menor nivel, formando así una jerarquía de activos anidados.

Adicionalmente a la introducción de los instrumentos financieros, el sistema debe incluir un mecanismo que permita calcular el valor del portfolio del cliente, entendiendo como portfolio la lista de productos que un cliente ha comprado.

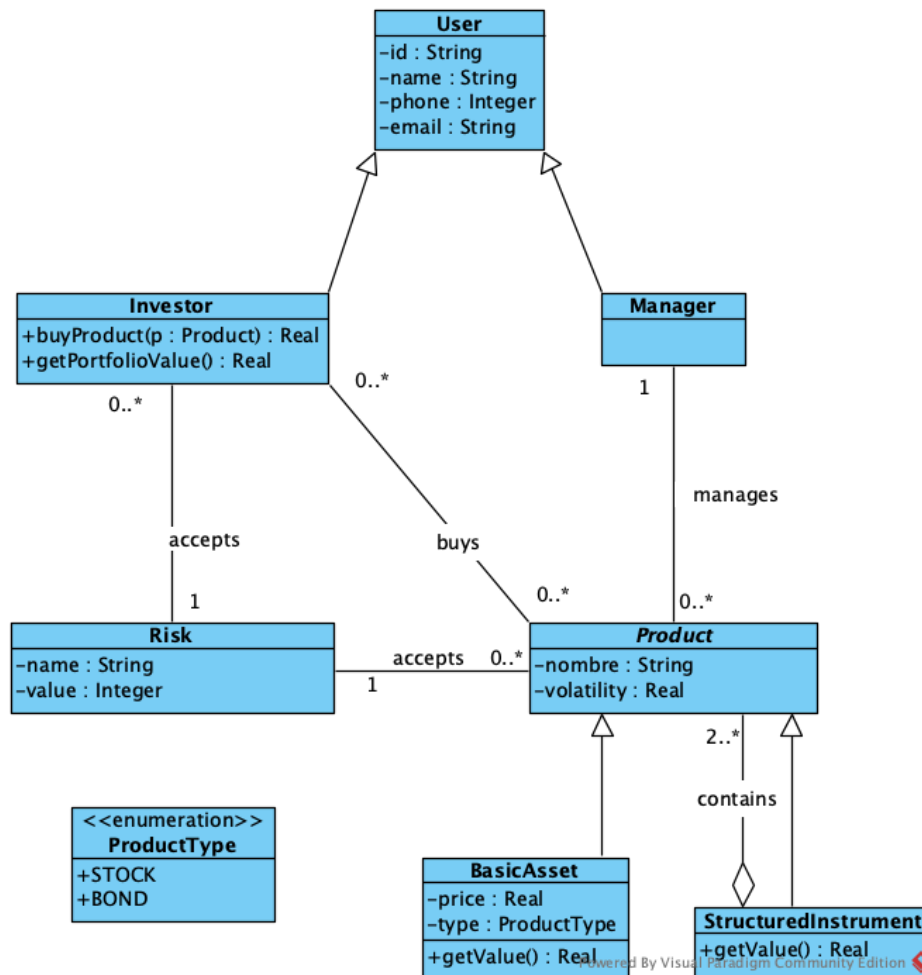
Pregunta 3.1. (5 %)

¿Qué patrón o patrones de análisis de los vistos en la asignatura serían interesantes utilizar para poder implementar esta funcionalidad?

Para resolver los requisitos planteados en la evolución del catálogo de servicios de InvestUOC, debe aplicarse el patrón **objeto compuesto (composite)**. El sistema necesita agrupar objetos básicos, como son los activos individuales, en contenedores (instrumentos estructurados), y permitir que estos contenedores incluyan a su vez otros contenedores (estrategias de inversión). Dado los requisitos, el patrón objeto compuesto permite al cliente (en este caso, el sistema de cálculo del portfolio) tratar de manera idéntica a los objetos individuales (acciones, bonos) y a las composiciones de objetos (instrumentos estructurados).

Pregunta 3.2 (15 %)

Proporcionar el modelo de clases UML que implemente la funcionalidad descrita en el apartado anterior. No olvides añadir las restricciones de integridad. Utiliza el modelo inicial del enunciado como base para hacer el nuevo modelo. No hace falta explicar detalladamente cada elemento del diagrama, solamente los cambios principales.



Vamos a desglosar los cambios realizados en función de los principios de diseño corregidos:

- **Aplicación del patrón objeto compuesto:** La clase original *Product* se convierte en una clase abstracta. A su vez, se crean dos nuevas subclases: *BasicAsset* y *StructuredInstrument* (el objeto compuesto). A partir de *StructuredInstrument*, añadimos una relación de agregación desde dicha clase hacia la superclase abstracta *Product* con cardinalidad 2..* para que combine al menos dos activos, aunque 1..* también sería aceptable dependiendo de la lógica de negocio. Gracias a estos cambios, permitimos la recursividad que el enunciado nos pide.
- El cálculo del valor a partir del método *getValue() : Real* en la clase abstracta *Product* introduce un polimorfismo, ya que en *BasicAsset* este método devolverá simplemente su atributo *price*.

En cambio, la implementación en *StructuredInstrument* iterará sobre todos los *Product* que contiene sumando los resultados de sus respectivos métodos *getValue()*.

Es importante mencionar que se ha movido el atributo *price* a *BasicAsset*, ya que el precio de un *StructuredInstrument* suele ser un valor calculado derivado de sus componentes, no un atributo estático.

- Para cumplir con el cálculo del portfolio ("lista de productos que un cliente ha comprado"), la relación *buys* entre *Investor* y *Product* ya actúa como el portfolio del cliente. Para poder calcular el valor del portfolio, simplemente añadimos el método *calculatePortfolioValue(): Real* en la clase *Investor*, el cual sumará el *getValue()* de todos los productos vinculados a través de la relación *buys*.

Restricciones de integridad:

- Un *StructuredInstrument* no puede formar parte de sus propios componentes, ni directa ni indirectamente.

Pregunta 3.3 (10 %)

Proporciona el pseudocódigo de la funcionalidad que permite calcular el valor del portfolio del cliente, cumpliendo los principios. El pseudocódigo debe incluir todo el código relacionado con esta funcionalidad.

```
enum ProductType { STOCK, BOND }

abstract class Product {
    protected String nombre;
    protected double volatility;
    public abstract double getValue();
}

class BasicProduct extends Product {
    private double price;
    private ProductType type;
    public double getValue() { return this.price; }
}

class StructuredProduct extends Product {
    private List<Product> components;
    public void addComponent(Product p) { this.components.add(p); }
    public double getValue() {
        double totalValue = 0.0;
        for (Product p : components) {
            totalValue += p.getValue();
        }
        return totalValue;
    }
}

class Investor extends User {
    private List<Product> portfolio;
    public void buyProduct(Product p) { this.portfolio.add(p); }
    public double calculatePortfolioValue() {
        double totalPortfolioValue = 0.0;

        for (Product p : portfolio) {
            totalPortfolioValue += p.getValue();
        }

        return totalPortfolioValue;
    }
}
```

Pregunta 4 (20 %)

Partiendo del enunciado inicial. Tras la inclusión inicial de los productos de acciones (*Stock*) bonos (*Bond*), la plataforma InvestUOC decide añadir dos nuevos productos financieros para diversificar la oferta con dos tipos de productos nuevos:

- *CryptoAsset*: Hace referencia a activos digitales que operan sobre una red *blockchain*, el más típico de este tipo de activos es Bitcoin, Ethereum y Solana.
- *Deposit* (Depósito a plazo fijo): Es un producto bancario que se consume mucho en España ya que no tiene riesgo y tiene una fecha de vencimiento fija en la que se liquida el activo y se recupera la cantidad aportada más los beneficios generados.

Al haber ya 4 tipos de productos y con peculiaridades especiales, el equipo de desarrollo ha decidido añadir a la clase abstracta *Product* y crear una clase que hereda de *Product* para cada tipo de producto.

A continuación se muestra el pseudocódigo de cómo quedarían la clase *Product* y las clases para los diferentes tipos de producto:

```
public abstract class Product {
    protected Real price;
    public abstract Real getPrice();
    public abstract String getBlockchainAddress();
    public abstract Date getMaturityDate();
}

public class Stock extends Product {
    public Real getPrice() { return this.price * 1.02; }
    public String getBlockchainAddress() { return null; }
    public Date getMaturityDate() { return null; }
}

public class Bond extends Product {
    public Real getPrice() { return this.price * 1.01; }
    public String getBlockchainAddress() { return null; }
    public Date getMaturityDate() { return null; }
}

public class CryptoAsset extends Product {
    public String address;
    public Real getPrice() { return this.price; } // no tiene comisión
    public String getBlockchainAddress() { return address; }
    public Date getMaturityDate() { return null; }
}
```

```
public class Deposit extends Product {
    // En determinados momentos del día no se puede consultar
    // el valor del fondo y queda marcado como bloqueado.
    private boolean isLocked;

    public Real getPrice(){
        if (this.isLocked) {
            this.price
        }
        return this.price + this.price*1/365;
    }
    public String getBlockchainAddress() { return null; }
    public Date getMaturityDate() { return new Date("2029-01-01"); }
}
```

Pregunta 4.1. (2.5 %)

Indica si este diseño satisface o no el principio de segregación de interfaces. Razona tu respuesta.

El principio de segregación de interfaces establece que los clientes no tendrían que depender de operaciones que no utilizan.. En otras palabras, fomenta la creación de interfaces pequeñas y específicas para cada cliente en lugar de una única interfaz general que contenga todos los métodos posibles.

En el pseudocódigo del enunciado, la clase abstracta *Product* actúa como una interfaz que obliga a todas las clases hijas a implementar métodos que no tienen sentido para su naturaleza particular. Esto lo podemos comprobar con las clases *Stock* y *Bond*, las cuales implementan los métodos *getBlockchainAddress()* y *getMaturityDate()* devolviendo *null*, ya que las acciones y los bonos tradicionales no operan en blockchain ni tienen una fecha de vencimiento configurada (según la lógica implementada).

Además, la clase *CryptoAsset* implementa el método *getMaturityDate()* devolviendo *null*, puesto que las criptomonedas no son depósitos a plazo fijo que venzan. *Deposit* también implementa *getBlockchainAddress()* devolviendo *null* porque un depósito bancario tradicional no existe en una red blockchain.

Como podemos apreciar, todas las implementaciones innecesarias siguen el patrón de devolver *null* para cumplir con el contrato de *Product*, lo cual es un síntoma manifiesto de que se está violando el principio de segregación de interfaces.

Pregunta 4.2. (2.5 %)

Indica si este diseño satisface o no el principio de sustitución de Liskov. Razona tu respuesta.

El principio de sustitución de Liskov establece que las instancias de una subclase *C* tienen que ser sustituibles por instancias de las superclases de *C*.

En la [pregunta 4.1](#), hemos visto que el diseño del enunciado viola el principio de segregación de interfaces. Esto obliga a clases como *Stock* o *Bond* a implementar métodos como *getBlockchainAddress()* que devuelven *null*. Imaginemos que tenemos un sistema que procesa una lista de productos (*List<Product>*) y que asume que puede imprimir la longitud de la dirección de la blockchain. Al sustituir el *Product* genérico por un *Stock*, la llamada al método *product.getBlockchainAddress().length()* provocará un error *NullPointerException*, ya que la subclase no cumple el contrato esperado de la superclase. Tendríamos que comprobar el tipo de clase de cada objeto o añadir comprobaciones de nulos para asegurar que el programa funciona correctamente, lo cual viola el principio de sustitución de Liskov.

Pregunta 4.3. (15 %)

Proporciona pseudocódigo que arregle los dos principios vistos en las preguntas 4.1 y 4.2, si fuera necesario.

```
public abstract class Product {
    protected Real price;
    public abstract Real getPrice();
}

// Interfaz específica para activos en blockchain
public interface BlockchainAsset {
    public String getBlockchainAddress();
}

// Interfaz específica para activos con vencimiento a plazo
public interface TermAsset {
    public Date getMaturityDate();
}

public class Stock extends Product {
    public Real getPrice() { return this.price * 1.02; }
}

public class Bond extends Product {
    public Real getPrice() { return this.price * 1.01; }
}

public class CryptoAsset extends Product implements BlockchainAsset {
    private String address;
    public Real getPrice() { return this.price; }
    public String getBlockchainAddress() { return this.address; }
}

public class Deposit extends Product implements TermAsset {
    private boolean isLocked;
    public Real getPrice() {
        if (this.isLocked) {
            return this.price;
        }
        return this.price + (this.price * 1/365);
    }
    public Date getMaturityDate() { return new Date("2029-01-01"); }
}
```